PROJECT MILESTONE DOCUMENTATION

Milestone 3: Budget, Expenses & Collaboration

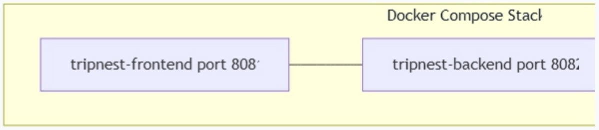
Project: TripNest Travel Platform

Scope: Budget Management, Group Collaboration & Deployment

This document covers the implementation, architecture, database schemas, APIs, and design of the collaboration and financial management features introduced in Milestone 3. The system builds on the Milestone 2 trip and itinerary core, adding expense tracking, group member management, group chat, travel document storage, and in-app notifications — all containerized with Docker Compose for full-stack deployment.

1. System Architecture

The architecture extends the Milestone 2 travel planning system with five new feature pillars — each backed by a dedicated Spring Boot service layer, repository, and React frontend component.



---

2. Database Schema Model

Hibernate automatically builds and configures the following five new tables based on the JPA entities introduced in Milestone 3. All tables cascade correctly from the `trips` table established in Milestone 2.

```
-- trip_members: Links users to trips as collaborators
CREATE TABLE trip_members (
  id         BIGINT PRIMARY KEY AUTO_INCREMENT,
  trip_id    BIGINT NOT NULL,
  user_id    BIGINT NOT NULL,
  role       VARCHAR(20) NOT NULL DEFAULT 'MEMBER',   -- MEMBER, OWNER
  status     VARCHAR(20) NOT NULL DEFAULT 'PENDING',  -- PENDING, ACCEPTED, DE
  invited_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  joined_at  DATETIME,
  UNIQUE KEY uq_trip_user (trip_id, user_id),
  FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- expenses: Tracks per-trip financial transactions
CREATE TABLE expenses (
  id           BIGINT PRIMARY KEY AUTO_INCREMENT,
  trip_id      BIGINT NOT NULL,
  user_id      BIGINT NOT NULL,
  category     VARCHAR(30) NOT NULL,
  -- TRANSPORTATION, HOTEL, FOOD, SHOPPING, ENTERTAINMENT, MISCELLANEOUS
  amount       DECIMAL(12, 2) NOT NULL,
  description  VARCHAR(255),
  expense_date DATE NOT NULL,
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- travel_documents: Stores uploaded file metadata per trip
CREATE TABLE travel_documents (
  id                BIGINT PRIMARY KEY AUTO_INCREMENT,
  trip_id           BIGINT NOT NULL,
  uploader_id       BIGINT NOT NULL,
  file_name         VARCHAR(255) NOT NULL,
  stored_file_name  VARCHAR(255) NOT NULL UNIQUE,
  doc_type          VARCHAR(30) NOT NULL,
  -- TICKET, HOTEL_BOOKING, PASSPORT, VISA, PHOTO, OTHER
  uploaded_at       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE,
  FOREIGN KEY (uploader_id) REFERENCES users(id)
);

-- group_messages: Stores per-trip group chat messages
CREATE TABLE group_messages (
CREATE TABLE group_messages (
  id         BIGINT PRIMARY KEY AUTO_INCREMENT,
  trip_id    BIGINT NOT NULL,
  sender_id  BIGINT NOT NULL,
  message    TEXT NOT NULL,
  sent_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE,
  FOREIGN KEY (sender_id) REFERENCES users(id)
);

-- notifications: Stores in-app notifications per user
CREATE TABLE notifications (
  id           BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id      BIGINT NOT NULL,
  type         VARCHAR(30) NOT NULL,
  -- TRIP_REMINDER, BUDGET_ALERT, GROUP_INVITE, ACTIVITY_REMINDER, GENERAL
  title        VARCHAR(150) NOT NULL,
  message      TEXT NOT NULL,
  is_read      BOOLEAN NOT NULL DEFAULT FALSE,
  reference_id BIGINT,
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

---

3. API Reference

`Authorization: Bearer <token>` rom the configured upload directory.

Trip Members & Collaboration

GET `/api/trips/{tripId}/members`

Returns all member records for a trip including their status (PENDING, ACCEPTED, DECLINED) and role.

POST `/api/trips/{tripId}/members/invite`

Invites a user by username or email. Only the trip owner may invite. Creates a PENDING member record and sends a GROUP_INVITE notification to the invitee.

PUT `/api/trips/{tripId}/members/{id}/accept`
Accepts a pending invitation. Updates member status to ACCEPTED and records the joined timestamp.

PUT `/api/trips/{tripId}/members/{id}/decline`
Declines a pending invitation. Updates member status to DECLINED.

DELETE `/api/trips/{tripId}/members/{id}`
Removes a member from the trip. Allowed for the trip owner (remove anyone) or the member themselves (leave trip).

Budget & Expenses

GET `/api/trips/{tripId}/expenses`
Returns all expenses for a trip ordered by date descending.

GET `/api/trips/{tripId}/expenses/summary`
Returns a budget summary: total budget, total spent, remaining balance, over-budget flag, and a per-category breakdown map.

POST `/api/trips/{tripId}/expenses`
Records a new expense. Body requires `category`, `amount`, and `expenseDate`.

PUT `/api/trips/{tripId}/expenses/{id}`
Updates an existing expense. Only the expense creator may edit.

DELETE `/api/trips/{tripId}/expenses/{id}`
Deletes an expense record. Only the expense creator may delete.

Travel Documents

GET `/api/trips/{tripId}/documents`
Lists all documents uploaded for a trip with metadata (file name, type, uploader, upload date, download URL).

POST `/api/trips/{tripId}/documents`
Uploads a file via `multipart/form-data`. Accepted types: TICKET, HOTEL_BOOKING, PASSPORT, VISA, PHOTO, OTHER. Max size: 10 MB.

GET `/api/trips/{tripId}/documents/{id}/download`

`/api/trips/{tripId}/documents/{id}/download`
Streams file bytes for download with correct `Content-Disposition` and `Content-Type` headers.

DELETE `/api/trips/{tripId}/documents/{id}`
Deletes the metadata record and removes the physical file from the server. Only the uploader may delete.

Group Chat

GET `/api/trips/{tripId}/messages`
Returns all group chat messages for the trip ordered chronologically, including sender username and timestamp.

POST `/api/trips/{tripId}/messages`
Posts a new message to the trip group chat. Body requires a non-empty `message` string.

Notifications

GET `/api/notifications`
Returns all in-app notifications for the authenticated user ordered by creation date descending.

GET `/api/notifications/unread-count`
Returns the count of unread notifications as `{ "count": N }` — used by the notification bell badge.

PUT `/api/notifications/{id}/read`
Marks a single notification as read. Only the notification owner may perform this action.

PUT `/api/notifications/read-all`
Marks all notifications for the authenticated user as read in a single bulk operation.

---

4. Frontend Component Breakdown

All Milestone 3 features are accessed through a tabbed interface inside the existing `TripDetail.js` page, extended with four new tabs: **Budget & Expenses**, **Members**,

Documents, and **Group Chat**. The notification bell is mounted globally in the Navbar component.

A. Budget & Expenses Tab (`BudgetOverview.js` + `ExpenseList.js` + `ExpenseForm.js`)

Presents a three-card header row showing Total Budget, Total Spent, and Remaining / Over Budget. A progress bar with a percentage indicator visually represents budget utilisation. A colour-coded category breakdown grid shows spending per category with individual progress bars.

- The expense table supports category filtering via toggle buttons and shows who recorded each expense.
- Each row displays category icon, description, date, amount (₹ formatted), and the recorder's username.
- Expense creators can edit or delete their own entries via an inline slide-up form.

B. Members Tab (`TripMembers.js`)

Provides the complete trip collaboration workflow. The trip owner sees an Invite Collaborator panel with a helper hint reminding that the invitee must be a registered TripNest user.

- Status-aware error messages distinguish between "user not found", "already invited", and "cannot invite yourself" cases.
- Accepted members are displayed as avatar cards with username, email, role, and a coloured status badge (Active / Pending / Declined).
- The trip owner can remove any member; members can leave using the same action.
- `TripService.getTripById()` was updated to allow accepted collaborators to access trip details alongside the owner.

C. Documents Tab (`DocumentManager.js`)

A drag-and-drop file upload interface where users select a document type before uploading. Files are stored with a UUID prefix to prevent naming collisions.

- Documents are grouped by type with distinct icons: 🎫 Ticket, 🏈 Hotel Booking, 🔘 Passport, 📋 Visa, 📷 Photo, 📄 Other.
- Each card shows the filename, uploader username, upload date, a download button, and a delete button (uploader only). Max file size: 10 MB.

and a delete button (uploader only). Max file size 10 MB.

D. Group Chat Tab (`GroupChat.js`)

A scrollable message thread grouped by calendar date. Messages appear as chat bubbles — right-aligned for the current user and left-aligned for others, each with a sender avatar initial.

- Polls for new messages every 5 seconds using `setInterval`, only triggering a re-render when the message list changes.
- New messages auto-scroll the view to the bottom.

E. Notification Bell (`NotificationBell.js`)

Mounted in the Navbar and visible across all authenticated pages. Polls every 30 seconds and displays a numeric badge for unread notifications.

- Opens a dropdown listing the 15 most recent notifications with type icons (📅 Trip Reminder, 💸 Budget Alert, 👥 Group Invite, ⏰ Activity Reminder, 🔔 General) and relative timestamps.
- Clicking an unread notification marks it read inline. "Mark all read" handles bulk dismissal.

---

5. Entity Relationship Overview

The ER diagram below shows the five new Milestone 3 entities and how they relate to the existing `users` and `trips` tables from Milestones 1 and 2.

6. Docker Deployment Architecture

Milestone 3 introduces full Docker Compose containerisation, enabling a single-command deployment of the complete three-tier application stack.

Container	Image	Host Port	Container Port	Purpose
`tripnest-db`	mysql:8	3307	3306	MySQL database with persistent named volume `db_data`
`tripnest-backend`	Built from `./tripnest/Dockerfile`	8082	8080	Spring Boot API — depends on DB health check before starting
`tripnest-frontend`	`./tripnest-frontend/Dockerfile`	8081	80	React app served by Nginx; proxies `/api/**` to backend

The backend Dockerfile uses a two-stage build: Stage 1 compiles the Spring Boot JAR inside a Maven container; Stage 2 copies only the JAR into a lightweight JRE Alpine image. A named Docker volume (`uploads_data`) is mounted at `/uploads` inside the backend container to persist travel documents across restarts.

```
# Run the full stack (first time or after source changes):
docker compose up --build

# Rebuild only backend and frontend (no DB reset):
docker compose build --no-cache backend frontend
docker compose up -d --no-deps backend frontend

# View running containers:
```

```
# View running containers:
docker compose ps

# Access the application:
#   Frontend  -> http://localhost:8081
#   Backend   -> http://localhost:8082/api/...
#   Database  -> localhost:3307
```

---

7. Summary of Completed Operations

Backend (Spring Boot)

- **CollaborationService:** Implemented invite-by-username-or-email with duplicate and self-invite guards. Auto-creates a `GROUP_INVITE` notification on successful invite.
- **ExpenseService:** Full CRUD with per-category aggregation query for the budget summary. Budget remaining and over-budget flag computed server-side.
- **DocumentService:** File upload via Spring `MultipartFile`. Files stored on disk with UUID prefix; metadata persisted in `travel_documents`. Download streams bytes with correct headers.
- **GroupChatService:** Append-only message persistence fetched in chronological order per trip.
- **NotificationService:** Internal creation API consumed by other services. Supports per-user read/unread state and bulk mark-all-read via a custom JPQL update query.
- **GlobalExceptionHandler:** Extended to handle `ResponseStatusException` — returns a structured `{ "message": "..." }` JSON body for all 4xx/5xx errors so frontend messages are always actionable.
- **TripService (fix):** `getTripById()` updated to grant access to both trip owners and accepted collaborators, unblocking the collaboration workflow.

Frontend (React)

- **TripDetail.js:** Extended with four new tabs — Budget & Expenses, Members, Documents, Group Chat — all lazy-rendered on tab activation.
- **BudgetOverview.js:** Real-time budget progress bar and category breakdown grid with colour-coded bars per expense category.
- **ExpenseList.js + ExpenseForm.js:** Filterable expense table with inline add/edit/delete

and owner-scoped permissions.

- **TripMembers.js:** Full invite lifecycle UI with status badge display and context-aware error messages.
- **DocumentManager.js:** Drag-and-drop upload zone with document type selector, grouped file listing, and per-file download/delete actions.
- **GroupChat.js:** Date-grouped chat bubble UI with 5-second polling and auto-scroll-to-bottom behaviour.
- **NotificationBell.js:** Navbar-mounted dropdown with unread badge, 30-second polling, relative timestamps, and mark-read interactions.

> **Evaluation Validation Note**
>
> All three Milestone 3 evaluation criteria are satisfied: **Budget management** is operational with full CRUD and live summary; **Expense tracking** is functional with category filtering and per-user ownership; **Collaboration workflow** is complete with invite, accept/decline, group chat, document management, and notifications — all containerized and running at `localhost:8081` via Docker Compose.